

Planejamento das Sprints

Interpretador de um subconjunto de C — Compiladores 1 · Grupo 10 · 2026.2

Metodologia: Scrum/XP, sprints de uma semana (12/08/2026 a 02/12/2026).

Situação atual (03/09/2026): Sprint 4 em andamento (02/09 a 09/09) — refinamento do parser e tratamento de erros sintáticos.

Legenda de status nas tarefas: [] pendente · [x] concluído.

Sumário

- Visão geral
- Calendário de aulas e eventos
- Marcos de avaliação
- Detalhamento das sprints

Visão geral

Sprint	Período	Foco	Aulas / eventos	Entregas-chave
1	12/08 – 19/08	Escopo da linguagem, setup e análise léxica inicial	12/08 avaliação diagnóstica · 17/08 linguagens formais · 19/08 prática de scanners	Repo GitHub · lexer.l inicial · README
2	19/08 – 26/08	Léxico completo e gramática inicial no Bison	24/08 estrutura de compiladores · 26/08 prática da fase léxica	lexer.l completo · parser.y inicial · Makefile/CMake
3	26/08 – 02/09	Análise sintática básica	31/08 GLC · 02/09 prática do parser	Gramática de declarações/expressões · testes.c · main.c
4	02/09 – 09/09	Parser completo e erros sintáticos	07/09 feriado · 09/09 prática de erros sintáticos	if/else/while/blocos · yyerror() com linha e coluna
5	09/09 – 16/09	AST e tabela de símbolos	14/09 AST e TS · 16/09 prática de AST/TS	ast.c/ast.h · symbol_table.c/symbol_table.h

Sprint	Período	Foco	Aulas / eventos	Entregas-chave
6	16/09 – 23/09	Im- pressão da AST e preparaçã do P1	21/09 e 23/09 Semana Universitária (sem aulas expositivas)	print_ast() · formulário P1 (até 23/09) · slides
7	23/09 – 30/09	Apresen- tação P1 e ajustes	28/09 P1 1ª parte · 30/09 P1 2ª parte	Apresentação P1 · relatório de débitos técnicos
8	30/09 – 07/10	Análise semân- tica na AST	05/10 análise semântica · 07/10 prática de semântica	semantic.c/semantic.h · checagem de tipos e escopos
9	07/10 – 14/10	Interpre- tador (exe- cução direta da AST)	12/10 feriado · 14/10 prática (adaptada ao interpretador)	interpreter.c/interpreter.h · ambiente de runtime
10	14/10 – 21/10	Otimiza- ção na interpre- tação da AST	19/10 otimização · 21/10 prática de otimização	optimizer.c/optimizer.h · testes comparativos
11	21/10 – 28/10	Fluxo de controle e progra- mas comple- tos	26/10 geração de código · 28/10 prática (execução direta)	if/while no interpretador · 10+ testes .c
12	28/10 – 04/11	Consoli- dação e preparaçã do P2	02/11 feriado · 04/11 Compiladores e LLM	Interpretador estável · formulário P2 (até 04/11) · slides
13	04/11 – 11/11	Apresen- tação P2 e backlog final	09/11 P2 1ª parte · 11/11 P2 2ª parte	Apresentação P2 · lista de melhorias finais
14	11/11 – 18/11	Poli- mento, escopos e extras	16/11 e 18/11 implementação final	Runtime errors · escopo de blocos · código limpo

Sprint	Período	Foco	Aulas / eventos	Entregas-chave
15	18/11 – 25/11	Code freeze e documentação final	23/11 e 25/11 implementação final / entrega no GitHub	Repo congelado · README completo · .zip se solicitado
16	25/11 – 02/12	Entrevistas finais	30/11 e 02/12 entrevistas presenciais	Defesa do projeto por todos os membros

Calendário de aulas e eventos

Aulas da disciplina caem em **segunda** e **quarta**.

Data	Dia	Tipo	Evento	Sprint
12/08	Qua	Aula	Avaliação diagnóstica	1
17/08	Seg	Aula	Fundamentos de linguagens formais	1
19/08	Qua	Prática	Construção de analisadores léxicos simples	1 → 2
24/08	Seg	Aula	Estrutura de compiladores e interpretadores	2
26/08	Qua	Prática	Projeto inicial do compilador — fase léxica	2 → 3
31/08	Seg	Aula	Análise sintática: gramáticas livres de contexto	3
02/09	Qua	Prática	Implementação inicial do parser	3 → 4
07/09	Seg	Feriado	Independência do Brasil — sem aula	4
09/09	Qua	Prática	Continuação do parser e tratamento de erros sintáticos	4 → 5
14/09	Seg	Aula	AST e tabela de símbolos	5

Data	Dia	Tipo	Evento	Sprint
16/09	Qua	Prática	Construção da AST e da tabela de símbolos	5 → 6
21/09	Seg	Recesso	Semana Universitária — sem aulas expositivas	6
23/09	Qua	Recesso / prazo	Semana Universitária · prazo do formulário P1	6 → 7
28/09	Seg	Avaliação	P1 — 1ª parte (apresentação, 5 min)	7
30/09	Qua	Avaliação	P1 — 2ª parte	7 → 8
05/10	Seg	Aula	Análise semântica	8
07/10	Qua	Prática	Integração da análise semântica ao compilador	8 → 9
12/10	Seg	Feriado	Nossa Senhora Aparecida — sem aula	9
14/10	Qua	Prática	Protótipo de geração de código intermediário (adaptado: interpretador da AST)	9 → 10
19/10	Seg	Aula	Otimização de código	10
21/10	Qua	Prática	Otimização de código	10 → 11
26/10	Seg	Aula	Geração de código final	11
28/10	Qua	Prática	Protótipo de geração de código final (adaptado: execução direta do programa)	11 → 12
02/11	Seg	Feriado	Finados — sem aula	12
04/11	Qua	Aula / prazo	Compiladores e LLM · prazo do formulário P2	12 → 13
09/11	Seg	Avaliação	P2 — 1ª parte	13
11/11	Qua	Avaliação	P2 — 2ª parte	13 → 14

Data	Dia	Tipo	Evento	Sprint
16/11	Seg	Atividade	Implementação final do compilador/interpretador	14
18/11	Qua	Atividade	Implementação final do compilador/interpretador	14 → 15
23/11	Seg	Atividade	Implementação final — entrega no GitHub	15
25/11	Qua	Atividade / prazo	Implementação final — code freeze e entrega no GitHub	15 → 16
30/11	Seg	Entrevista	Entrevistas de entrega do trabalho final (presencial)	16
02/12	Qua	Entrevista	Entrevistas de entrega do trabalho final (presencial)	16

Marcos de avaliação

Marco	Data	Sprint	O que entregar
Formulário P1	até 23/09	6	Formulário P1 preenchido pelo líder
Apresentação P1	28/09 ou 30/09	7	Slides (5 min) + demo do pipeline até a AST
Formulário P2	até 04/11	12	Formulário P2: avanços desde o P1, execução da AST, otimizações
Apresentação P2	09/11 ou 11/11	13	Demo ao vivo do interpretador executando programas C
Entrega no GitHub / code freeze	25/11	15	Repositório finalizado 15 dias antes das entrevistas
Entrevistas finais	30/11 e 02/12	16	Defesa presencial — todos os membros

Detalhamento das sprints

Sprint 1 — Definição do escopo, setup e análise léxica inicial

Campo	Conteúdo
Período	12/08 a 19/08
Foco	Definição do escopo da linguagem, setup do ambiente e análise léxica inicial
Aulas	12/08 avaliação diagnóstica · 17/08 fundamentos de linguagens formais · 19/08 prática: analisadores léxicos simples

Objetivos

#	Objetivo
1	Organizar a equipe, criar o repositório e configurar o ambiente de compilação
2	Definir o subconjunto da linguagem C aceito pelo interpretador
3	Implementar o primeiro protótipo do scanner no Flex

Entregas

#	Entrega
1	Repositório GitHub estruturado, com acesso concedido ao professor
2	Arquivo <code>lexer.l</code> inicial reconhecendo os primeiros tokens
3	Estrutura inicial do arquivo de documentação <code>README.md</code>

Tarefas

Status	Frente	Atividade
[]	Especificação da linguagem	Definir o subconjunto de C suportado: tipos <code>int</code> e <code>float</code> ; operadores <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>=</code> , <code>≠</code> , <code><</code> , <code>></code> ; controle <code>if</code> , <code>else</code> , <code>while</code> ; E/S básica com <code>printf</code>
[]	Repositório	Criar o repositório no GitHub, configurar <code>.gitignore</code> para C/Flex/Bison e convidar o professor (sergioaafreitas)
[]	Ambiente	Garantir que todos instalem e testem GCC/Clang, Make/CMake, Flex e Bison
[]	Léxico (<code>lexer.l</code>)	ERs para palavras-chave: <code>int</code> , <code>float</code> , <code>if</code> , <code>else</code> , <code>while</code> , <code>return</code>

Status	Frete	Atividade
[]	Léxico (lexer.l)	ERs para identificadores ([a-zA-Z_][a-zA-Z0-9_]*) e inteiros ([0-9]+)
[]	Léxico (lexer.l)	Contador de linhas e colunas para tratamento posterior de erros
[]	README	Estrutura do projeto: pastas /src, /include, /tests
[]	README	Planejamento das sprints: escopo e tarefas da Sprint 1

Sprint 2 — Léxico completo e gramática inicial no Bison

Campo	Conteúdo
Período	19/08 a 26/08
Foco	Finalização da análise léxica e início da gramática no Bison
Aulas	24/08 estrutura de compiladores e interpretadores · 26/08 prática: projeto inicial do compilador (fase léxica)

Objetivos

#	Objetivo
1	Concluir a especificação dos tokens no Flex (caracteres especiais e comentários)
2	Criar o arquivo do Bison com a estrutura inicial de tipos e regras

Entregas

#	Entrega
1	lexer.l completo e imune a erros de caracteres desconhecidos
2	parser.y inicial compilando via bison -d e gerando parser.tab.h
3	Makefile ou CMakeLists.txt automatizando o build

Tarefas

Status	Frente	Atividade
[]	Léxico avançado	Regras para literais de ponto flutuante ($[0-9]^+\backslash.[0-9]^+$)
[]	Léxico avançado	Ignorar espaços ($[\backslash t\backslash r\backslash n]^+$) e comentários de linha ($// \dots$) e de bloco ($/* \dots */$)
[]	Léxico avançado	Tokens compostos: $\leq$, $\geq$, $=$, $\neq$, $\&\&$, $\backslash\backslash\backslash$
[]	Léxico avançado	Regra genérica de erro léxico para caracteres inválidos
[]	Integração Flex–Bison	Criar <code>parser.y</code> com <code>%token</code> para cada token do Flex
[]	Integração Flex–Bison	Definir <code>%union</code> (ou integração C++) para repassar valores em <code>yyval</code>
[]	Build	Script Makefile para Flex + Bison + GCC com um único <code>make</code>
[]	README	Decisões técnicas: padrão de arquivos, organização dos tokens e ferramentas
[]	README	Problemas e soluções: tratamento de comentários em bloco no Flex

Sprint 3 — Análise sintática básica

Campo	Conteúdo
Período	26/08 a 02/09
Foco	Construção da análise sintática básica
Aulas	31/08 análise sintática (GLC) · 02/09 prática: implementação inicial do parser

Objetivos

#	Objetivo
1	Desenvolver as regras gramaticais básicas da linguagem C no Bison
2	Resolver ambiguidades de precedência de operadores

Entregas

#	Entrega
1	Gramática em parser . y reconhecendo declaração de variáveis, expressões aritméticas e atribuições
2	Conjunto de arquivos de teste . c de validação sintática

Tarefas

Status	Frente	Atividade
[]	Gramática	Regra raiz program → declaration_list
[]	Gramática	Declaração de variáveis: type id; e type id = expr;
[]	Gramática	Expressões aritméticas (+, -, *, /) e agrupamento por parênteses
[]	Precedência	%left e %right no Bison (multiplicação priorizada sobre adição)
[]	Precedência	Resolver conflitos shift/reduce das regras de expressões
[]	Driver	main.c com main() que recebe um arquivo e invoca yyparse()
[]	README	Decisões técnicas: estrutura da BNF e regras de precedência no Bison

Sprint 4 — Parser completo e erros sintáticos

Campo	Conteúdo
Período	02/09 a 09/09
Foco	Refinamento do parser e tratamento de erros sintáticos
Aulas	07/09 feriado · 09/09 prática: continuação do parser e tratamento de erros sintáticos

Objetivos

#	Objetivo
1	Expandir a gramática para estruturas de controle (if/else, while) e blocos {}

Objetivo	
#	
2	Implementar relatório amigável de erros sintáticos com <code>yyerror()</code>

Entregas

Entrega	
#	
1	Gramática completa com blocos <code>{}</code> e estruturas condicionais e de repetição
2	Relatório de erros sintáticos indicando linha e coluna

Tarefas

Status	Frente	Atividade
☐	Gramática	Comandos compostos / blocos (<code>{ statement_list }</code>)
☐	Gramática	<code>if (expr) statement</code> e <code>if (expr) statement else statement</code> (dangling else com <code>%nonassoc</code>)
☐	Gramática	Laços <code>while (expr) statement</code>
☐	Erros	Customizar <code>yyerror(const char *s)</code> para: Erro Sintático na linha X, coluna Y: [descrição]
☐	Erros	<code>%define parse.error verbose</code> no Bison para mensagens mais detalhadas
☐	README	Problemas e soluções: resolução do dangling else no Bison

Sprint 5 — AST e tabela de símbolos

Campo	Conteúdo
Período	09/09 a 16/09
Foco	Construção da Árvore Sintática Abstrata (AST) e tabela de símbolos
Aulas	14/09 AST e tabela de símbolos · 16/09 prática: construção da AST e da tabela de símbolos

Objetivos

#	Objetivo
1	Criar as estruturas de dados dos nós da AST
2	Modificar o arquivo <code>.y</code> para construir a AST durante a análise sintática
3	Implementar a tabela de símbolos básica

Entregas

#	Entrega
1	Módulo <code>ast.h</code> / <code>ast.c</code> com nós de expressão, comando, atribuição e declaração
2	Módulo <code>symbol_table.h</code> / <code>symbol_table.c</code> com inserção e busca
3	Parser gerando a raiz da AST ao final de <code>yyparse()</code>

Tarefas

Status	Frente	Atividade
[]	Modelagem da AST	Enumeração <code>NodeType</code> : <code>NODE_VAR_DECL</code> , <code>NODE_ASSIGN</code> , <code>NODE_BIN_OP</code> , <code>NODE_IF</code> , <code>NODE_WHILE</code> , <code>NODE_LITERAL</code>
[]	Modelagem da AST	Estrutura de nós com ponteiros para filhos (esquerdo/direito ou lista) e dados específicos
[]	Construção da AST	Ações semânticas no Bison (ex.: <code>\$\$ = create_bin_op_node('+', \$1, \$3);</code>)
[]	Tabela de símbolos	Hash ou lista encadeada armazenando nome, tipo, endereço/escopo
[]	Tabela de símbolos	Funções <code>insert_symbol()</code> e <code>lookup_symbol()</code>
[]	README	Estrutura do projeto: novos módulos de AST e tabela de símbolos
[]	README	Decisões técnicas: arquitetura dos nós da AST

Sprint 6 — Visualização da AST e preparação do P1

Campo	Conteúdo
Período	16/09 a 23/09
Foco	Impressão da AST, consolidação léxica/sintática e preparação do P1
Aulas	21/09 e 23/09 Semana Universitária — sem aulas expositivas

Objetivos

#	Objetivo
1	Visualizar a AST gerada (depuração)
2	Consolidar os entregáveis e preencher o formulário do ponto de controle P1

Entregas

#	Entrega
1	Módulo de exibição textual/gráfica da AST (<code>print_ast(ASTNode* root)</code>)
2	Formulário de P1 preenchido pelo líder antes de 23/09
3	Apresentação em slides (5 minutos) pronta para 28/09 ou 30/09

Tarefas

Status	Frente	Atividade
☐	Visualizador	Impressão indentada da AST no terminal para depuração
☐	Testes	Suíte da fase front-end: 5+ programas em C cobrindo erros e geração de AST
☐	Formulário P1	Preencher o formulário P1: definição do projeto, linguagem e ferramentas, planejamento das sprints e funcionalidades já implementadas
☐	Apresentação	Material conciso do P1 demonstrando o pipeline até a AST funcionando
☐	README	Planejamento das sprints: planejado vs. executado até o marco P1

Sprint 7 — Apresentação P1 e ajustes de feedback

Campo	Conteúdo
Período	23/09 a 30/09
Foco	Apresentação do ponto de controle P1 e ajustes de feedback
Aulas	28/09 P1 1ª parte · 30/09 P1 2ª parte

Objetivos

#	Objetivo
1	Apresentar o progresso da equipe no ponto de controle P1
2	Absorver os feedbacks do professor e ajustar o backlog das sprints futuras

Entregas

#	Entrega
1	Apresentação presencial do P1 realizada
2	Relatório de correções e débitos técnicos pós-P1

Tarefas

Status	Frente	Atividade
[]	Apresentação P1	Apresentar equipe, motivação, arquitetura atual (Flex + Bison + AST) e executar um código demonstrativo ao vivo
[]	Refatoração	Atender às solicitações pontuais feitas pelo professor na arguição
[]	Front-end	Congelar léxico, sintático e criação da AST 100% estáveis antes da semântica
[]	README	Problemas e soluções: observações do professor no P1 e plano de ação

Sprint 8 — Análise semântica integrada à AST

Campo	Conteúdo
Período	30/09 a 07/10

Campo	Conteúdo
Foco	Análise semântica integrada à AST
Aulas	05/10 análise semântica (teórica) · 07/10 prática: integração da análise semântica

Objetivos

Objetivo

- 1 Implementar a verificação semântica percorrendo a AST
- 2 Garantir checagem de tipos e validação de escopos

Entregas

Entrega

- 1 Módulo `semantic.h` / `semantic.c` com o *semantic walk* na AST
- 2 Mensagens de erro semântico claras, emitidas antes da execução

Tarefas

Status	Frente	Atividade
[]	Declaração / uso	Detectar uso de variáveis não declaradas (<i>Undeclared Variable Error</i>)
[]	Declaração / uso	Detectar redeclaração no mesmo escopo (<i>Redeclaration Error</i>)
[]	Tipos	Validar atribuições incompatíveis (ex.: <code>string</code> /ponteiro em <code>int</code>)
[]	Tipos	Coerção implícita simples quando permitida (ex.: <code>int</code> → <code>float</code> em operações mistas)
[]	Controle de fluxo	Condições de <code>if</code> e <code>while</code> avaliáveis como booleanas/inteiras
[]	README	Decisões técnicas: tabela de compatibilidade de tipos e regras semânticas

Sprint 9 — Interpretador (execução direta da AST)

Campo	Conteúdo
Período	07/10 a 14/10
Foco	Construção do interpretador (execução direta da AST)
Aulas	12/10 feriado · 14/10 prática: protótipo de geração de código intermediário (adaptado para o interpretador da AST)

Objetivos

#	Objetivo
1	Substituir a geração de código intermediário pela execução direta da AST (Guia de Interpretadores)
2	Avaliar expressões e atribuições na memória de runtime

Entregas

#	Entrega
1	Módulo <code>interpreter.h</code> / <code>interpreter.c</code> executando expressões aritméticas e atribuições em tempo real
2	Estrutura de memória de runtime (ambiente/escopo de execução) com os valores atuais das variáveis

Tarefas

Status	Frente	Atividade
[]	Runtime	Estrutura <code>Environment</code> com pares (variável, valor atual)
[]	Avaliação	Função recursiva <code>LiteralValue evaluate_expr(ASTNode* node)</code>
[]	Avaliação	Resolver nós binários (+, -, *, /) calculando o resultado dinamicamente
[]	Atribuição	<code>NODE_ASSIGN</code> : avaliar o lado direito e atualizar o valor da variável do lado esquerdo
[]	README	Estrutura do projeto: módulo interpretador / runtime environment

Sprint 10 — Otimização na interpretação da AST

Campo	Conteúdo
Período	14/10 a 21/10
Foco	Otimização na interpretação da AST
Aulas	19/10 otimização de código (teórica) · 21/10 prática: otimização de código

Objetivos

#	Objetivo
1	Aplicar técnicas simples de otimização na AST antes da execução, para aumentar a eficiência do interpretador

Entregas

Entrega

- Módulo `optimizer.h / optimizer.c` com transformações na AST
- Testes comparativos mostrando a redução de nós após a otimização

Tarefas

Status	Frente	Atividade
[]	Constant folding	Percorrer a AST e, em operações binárias com dois literais (ex.: $3 + 5$), substituir o nó pelo literal do resultado (8)
[]	Simplificação	Eliminar expressões nulas/redundantes: $x + 0 \rightarrow x$, $x * 1 \rightarrow x$
[]	Dead code	Remover blocos <code>if (0) { ... }</code> que nunca executam
[]	README	Decisões técnicas: regras de constant folding e simplificação de expressões

Sprint 11 — Fluxo de controle e programas completos

Campo	Conteúdo
Período	21/10 a 28/10
Foco	Execução de fluxo de controle e teste de programas completos
Aulas	26/10 geração de código final (teórica) · 28/10 prática: protótipo de geração de código final (adaptado para execução direta)

Objetivos

#	Objetivo
1	Implementar a execução de laços (while) e condicionais (if/else) no interpretador
2	Rodar programas completos com loops e tomadas de decisão

Entregas

#	Entrega
1	Interpretador capaz de rodar scripts com condicionais e repetição completas
2	Suíte com 10+ programas .c de teste (fatorial, primos, Fibonacci, etc.)

Tarefas

Status	Frente	Atividade
[]	Condicionais	NODE_IF: avaliar a condição; se verdadeira, execute_ast(node→then_branch); senão, execute_ast(node→else_branch)
[]	Laços	NODE_WHILE: while (evaluate_expr(node->condition)) executando o bloco interno recursivamente
[]	E/S	printf() lendo o valor atual das variáveis no ambiente runtime e imprimindo no stdout
[]	README	Problemas e soluções: controle de pilha/recursão em laços profundos

Sprint 12 — Consolidação do interpretador e preparação do P2

Campo	Conteúdo
Período	28/10 a 04/11
Foco	Consolidação do interpretador e preparação para o P2
Aulas	02/11 feriado · 04/11 Compiladores e LLM

Objetivos

#	Objetivo
1	Estabilizar todas as funcionalidades do interpretador
2	Preencher o formulário do ponto de controle P2

Entregas

#	Entrega
1	Interpretador funcional e testado em diversos cenários
2	Formulário de P2 preenchido e enviado antes de 04/11
3	Apresentação do P2 estruturada e treinada

Tarefas

Status	Frente	Atividade
[]	Qualidade	Varredura de vazamentos de memória com Valgrind durante o percurso da AST
[]	Formulário P2	Preencher: principais avanços desde o P1, demonstração da execução da AST, otimizações e ajustes no planejamento
[]	Apresentação P2	Focar na interpretação em tempo de execução e na execução dos códigos de teste
[]	README	Planejamento das sprints: fechamento do status até a Sprint 12

Sprint 13 — Apresentação P2 e refinamento do backlog

Campo	Conteúdo
Período	04/11 a 11/11
Foco	Apresentação do ponto de controle P2 e refinamento do backlog
Aulas	09/11 P2 1ª parte · 11/11 P2 2ª parte

Objetivos

#	Objetivo
1	Apresentar o ponto de controle P2 ao professor
2	Mapear e implementar os ajustes solicitados na avaliação

Entregas

#	Entrega
1	Apresentação presencial do P2 concluída
2	Lista de melhorias finais prioritárias para a entrega do projeto

Tarefas

Status	Frente	Atividade
[]	Apresentação P2	Demo ao vivo do interpretador rodando programas C e exibindo os resultados calculados
[]	Feedback	Identificar bugs de execução ou casos não previstos na gramática/semântica apontados pelo professor
[]	Backlog	Priorizar as tarefas restantes para garantir zero erros na entrega definitiva
[]	README	Problemas e soluções: feedbacks do P2 e alterações realizadas no código

Sprint 14 — Polimento, escopos avançados e recursos extras

Campo	Conteúdo
Período	11/11 a 18/11
Foco	Polimento, escopos avançados e recursos adicionais

Campo	Conteúdo
Aulas	16/11 e 18/11 atividade: implementação final do compilador/interpretador

Objetivos

#	Objetivo
1	Incluir recursos extras se o cronograma permitir (funções simples ou arrays)
2	Garantir mensagens de erro e exceções de runtime amigáveis (ex.: divisão por zero)

Entregas

#	Entrega
1	Tratamento de exceções em tempo de execução (divisão por zero, variáveis <i>uninitialized</i>)
2	Código-fonte refatorado e limpo

Tarefas

Status	Frente	Atividade
[]	Runtime errors	Na divisão, emitir Runtime Error: Division by zero e abortar suavemente
[]	Escopo de blocos	Variáveis declaradas em {} de if/while destruídas ao sair do bloco
[]	Limpeza	Padronizar nomes, remover printf de depuração e organizar arquivos .h / .c
[]	README	Decisões técnicas: arquitetura de escopos de memória de runtime

Sprint 15 — Code freeze e documentação final

Campo	Conteúdo
Período	18/11 a 25/11

Campo	Conteúdo
Foco	Congelamento do código (<i>code freeze</i>) e fechamento da documentação final
Aulas	23/11 e 25/11 atividade: implementação final — entrega no GitHub

Objetivos

#	Objetivo
1	Congelar o código-fonte 15 dias antes das entrevistas (exigência do professor)
2	Concluir a documentação completa no GitHub

Entregas

#	Entrega
1	Repositório no GitHub finalizado e atualizado
2	Manual de uso e guia de compilação no README.md
3	Arquivo .zip com código e documentação (caso solicitado via Teams)

Tarefas

Status	Frente	Atividade
☐	Code freeze	Commit final da versão estável do interpretador
☐	Repositório	Verificar permissão de leitura do professor e se a estrutura atende a todos os critérios
☐	README	Estrutura do projeto: mapeamento completo de pastas, arquivos e módulos
☐	README	Decisões técnicas: resumo da Sprint 1 à 15 (Flex, Bison, AST, tabela de símbolos, interpretação direta e otimizações)
☐	README	Planejamento das sprints: histórico consolidado do que foi executado
☐	README	Problemas e soluções: tabela/lista de bugs e desafios da equipe e como foram resolvidos

Status	Frente	Atividade
[]	Guia	Tutorial passo a passo: clonar, compilar (make) e rodar scripts de teste no interpretador

Sprint 16 — Entrevistas finais e encerramento

Campo	Conteúdo
Período	25/11 a 02/12
Foco	Preparação para as entrevistas finais e encerramento da disciplina
Aulas	30/11 e 02/12 entrevistas de entrega do trabalho final (presencial)

Objetivos

Objetivo

- 1 Alinhar o conhecimento entre todos os membros da equipe
- 2 Participar da entrevista presencial com o professor

Entregas

#	Entrega
1	Entrevista de defesa do projeto final realizada por todos os membros da equipe

Tarefas

Status	Frente	Atividade
[]	Alinhamento	Reunião interna: cada membro explica a parte que desenvolveu e tira dúvidas. Todos devem dominar do Flex até a execução da AST
[]	Demonstração	Notebooks preparados com o interpretador pronto para compilar e executar na frente do professor

Status	Frente	Atividade
[]	Entrevista	Apresentar o interpretador funcionando, responder perguntas técnicas sobre as escolhas de projeto e demonstrar a robustez do software